NIM sizing API: model to recommendation

Recommendation API | Published profiles + installed hardware | No deployment or reservation

1. Recommendation request

Exact NIM image + tag and cluster ID
Optional GPU type
POST right-sizing/recommendation

2. Check the request

Existing authentication and access checks
Validate cluster ID, image, tag and type
catalog-service owns the sizing endpoint

3. Read cluster inventory

cluster-service: every inventory page
Separate GPU types and capacities
Matching device count on one node

4. Get exact-tag NGC profiles

Reuse successful, nonempty metadata
Existing CP cache: 15-minute default
Otherwise fetch public NGC metadata

5. Find applicable profiles

Direct PCI, tested PCI, or GPU-agnostic
Explicit positive TP/PP and minimum
Memory and GPU count must fit one node

6. Choose a fitting profile

Prefer a single-GPU configuration
Then: hardware evidence, fewer GPUs, lower per-device minimum, profile ID

No eligible profile

7. Apply allocation policy

One GPU: below 90% -> memory minimum
90-100% -> one whole GPU
Parallel: exactly TP x PP whole GPUs

8. Return the JSON result

Profile ID, precision, backend, minimum
TP/PP layout and allocation
HTTP 200: recommended

Supported alternative

HTTP 200: no_recommendation + reason
Missing capacity/count or no fitting profile
No cluster GPUs: empty results

Hardware identity and tested-device lists

Only unambiguous exact names establish PCI identity.
Empty tested_gpu_devices does not exclude an untargeted profile.
Unknown/ambiguous names can still use a GPU-agnostic profile.

Policy and scope

90% is one configurable sharing policy; no added headroom.
Published minima are per device and depend on profile settings.
This response does not verify free GPUs, scheduling or startup.

Solid cards: implemented behavior. Blue: processing. Green: recommendation. Amber: valid alternative. Gray: context.

Request/authentication failures and dependency errors are shown in the API decision diagram.

NIM sizing API: decisions and responses

POST /api/v1/inference/right-sizing/recommendation | Existing authentication applies first

1. Authorized request with valid inputs?
Required: exact NIM image, separate tag and cluster ID.
Optional gpuType must be nonempty when provided.

No → **Request rejected**
Existing authentication/access errors
Invalid input: HTTP 400

Yes

2. Retrieve fresh, paginated cluster inventory
Read each GPU type/capacity and its matching count on one node.

Error → **Inventory unavailable**
HTTP 503 dependency error

3. Is gpuType supplied?
Provided: exact inventory type only; evaluate each reported capacity.
Omitted/null: evaluate every reported type/capacity independently.
Provided but absent from inventory: HTTP 400.
Omitted/null with no inventory: HTTP 200 with results: [].

Absent → **Unknown explicit type**
HTTP 400 validation error

Empty → **No cluster GPUs**
HTTP 200 | results: []

Hardware selected

4. Get NGC profiles for this exact image + tag
Cache hit: reuse metadata. Cache miss/error: fetch NGC; default expiry is 15 minutes.
404 or empty metadata: continue with no profiles. Do not cache emptiness/errors.

Error → **NGC unavailable**
HTTP 503 dependency error
No expired-data fallback

5. For each selected type/capacity: a fitting applicable profile?
Require capacity, per-node count, explicit positive TP/PP and published per-device minimum.
Match direct PCI, tested PCI, or untargeted GPU-agnostic applicability.
Empty/missing tested_gpu_devices: an untargeted profile remains eligible.
Prefer one GPU; then hardware evidence, GPU count, minimum and profile ID.

No → **No recommendation**
HTTP 200 | no_recommendation
Return a clear per-hardware reason
No inferred GPU-count fallback

Yes

6. Does the selected profile use one GPU?
Count = published tensor parallel size x pipeline parallel size; respect max_gpu_count.

No → **Parallel allocation**
Exactly TP x PP whole GPUs
Full per-device capacity
Published minimum fits each GPU

Yes: below cutoff → **Memory allocation**
Below 90% of per-device capacity
Allocate the published memory minimum

Yes: cutoff through capacity → **One whole GPU**
90-100% of per-device capacity
Allocate the full device capacity

HTTP 200 | recommended
Return profile ID, precision, backend, published minimum, TP/PP and allocation. One request may mix result statuses.

90% is configurable and controls sharing. No added headroom. If a minimum exceeds capacity, skip that profile and consider other published layouts.

Solid cards: implemented behavior. Blue: processing. Green: recommended. Amber: valid alternative. Red: request/dependency failure.